

Aurora 900 SDK Developer guide

V1.7 - 03/04/2025

catalogue

1 Summary	6
2 Installation instructions	7
2.2 Aurora SDK Installation	7
2.2.1 Compile environment configuration	7
2.2.2 SDK depends on environment configuration	7
2.2.3 Other documentation	8
3 Aurora 900 SDK Call flow description	9
3.1 Interface call flow	9
3.2 Instructions for concurrent use	9
4 SDK usage example	10
4.1 Open the camera	10
4.2 Create/start the stream	11
4.3 Get the image	11
4.4 Close the video stream	12

4.5 Close the camera	12
5 SDK interface specification	13
5.1 Header notes	13
5.2 Equipment module description	13
5.2.1 Get the device manager object.....	13
5.2.2 Get the device list.....	13
5.2.3 Register device plug-in callback.....	14
5.2.4 Create a device object.....	14
5.2.5 Create the device via the USB port.....	15
5.2.6 Get the SDK version number	15
5.2.7 Get the device object type.....	15
5.2.8 Set the log output switch.....	15
5.2.9 Turn on the device	16
5.2.10 Get the supported image frame format.....	16
5.2.11 Get the supported stream types.....	17
5.2.12 Set the IR frame rate	17
5.2.13 Set the device image frame format.....	17
5.2.14 Close the device	18
5.2.15 Restart the device	18
5.2.16 Upgrade the equipment.....	18
5.2.17 Create a stream object.....	19
5.2.18 Destroy the object.....	19
5.2.19 Rgbdir Align switch (only supports AURORA930).....	19
5.2.20 Set the threshold size for noise removal	20
5.2.21 Obtain the serial number of the device product.....	20
5.2.22 Obtain the internal and external parameters of the camera	20
5.2.23 Get device information	21
5.2.24 Restart the device	21
5.2.25 Stop the device	22
5.2.26 Get the device status.....	22
5.2.27 Get device version information	22
5.2.28 Deep/infrared image mirroring (aurora900 equipment available)	23
5.2.29 Softreset device.....	23
5.2.30 Set the automatic exposure mode.....	23
5.2.31 Get the automatic exposure mode.....	24
5.2.32 Switch of the automatic exposure function.....	24

5.2.33 Set the exposure time	24
5.2.34 Get the exposure time	25
5.2.35 Set sensor gain	25
5.2.36 Get sensor gain	25
5.2.37 Set indoor and outdoor mode (only support AURORA930)	26
5.2.38 Set the depth filter range (only support AURORA930)	26
5.2.39 Switch of depth correction (only supported by AURORA930)	27
5.2.40 Set the face exposure area	27
5.2.41 Get information about device support	27
5.2.42 Get temperature information	28
5.2.43 Rollback firmware version	28
5.2.44 Set the depth of the edge trimming size	29
5.2.45 Set the depth distance	29
5.2.46 Set the heartbeat	29
5.2.47 Stop the heartbeat	30
5.2.48 Set it to scan code mode	30
5.2.49 Set the image resolution	30
5.2.50 Get the laser current	31
5.2.51 Set the laser current	31
5.2.52 Set Laser/Led light mode	32
5.3 Stream module description	32
5.3.1 Start the stream	32
5.3.2 Stop streaming	32
5.3.3 Get single/multi-frame images	33
5.3.4 Register the callback interface	33
6 Structure and enumeration type description	34
6.1 Basic type	34
6.1.1 Stream type	34
6.1.2 frame type	34
6.1.3 A single frame	34
6.1.4 Multi frames	35
6.1.5 facility information	36
6.2 Device type	36
6.2.1 Camera component type	36

6.2.2 Camera component ID	36
6.2.3 Facility information	37
6.2.4 Version information	37
6.2.5 Frame format	38
6.2.6 Camera resolution (RGB camera)	38
6.3 Depth calculation type	38
6.3.1 Camera intrinsic parameters	38
6.3.2 Camera Extrinsic parameters	39
6.3.3 Spatial point coordinates	39
6.3.4 Point cloud coordinates and corresponding rgb and ir data	39
Disclaimer	41
Technical support	41

1 Summary

Aurora SDK is a software development kit based on Deptrum Aurora RGBD modules, providing application developers with a series of user-friendly APIs and simple application examples. Users can leverage this development kit to obtain high-precision depth images, grayscale images, and color image information, facilitating the development of applications such as facial recognition, 3D reconstruction, and AI perception. This document introduces the usage methods of Aurora SDK (C++), including the usage process, detailed interface explanations, and example program usage.

2 Installation instructions

2.1.1.1 Hardware installation

The Aurora 3D camera is connected to the USB port of the PC through a cable.

In the Windows system, after the module is successfully connected, the Windows device manager appears "USB Lite-U20L", as shown in Figure 1,



Figure 1 Device name

Under Linux system, run "lsusb" in the terminal, and a device with vid 1900 appears, indicating that the module is successfully connected.

2.2 Aurora SDK Installation

2.2.1 Compile environment configuration

- Install the software package build-essential

1. `sudo apt-get install build-essential`

The build-essential function provides a list of the packages that the compiler must have.

- Install the cross-platform compiler tool CMake

1. `sudo apt-get install cmake`

Note: This SDK requires cmake 3.13.x or higher.

2.2.2 SDK depends on environment configuration

- Check or install libusb

1. `sudo apt-get install libusb-dev`
2. `sudo apt-get install libusb-1.0-0-dev`

- Check or install the opengl and glfw libraries

1. `sudo apt-get install libgl1-mesa-dev`
2. `sudo apt-get install build-dep glfw`
3. `sudo apt-get install libglu1-mesa-dev`
4. `sudo apt-get install libxi-dev`

2.2.3 Other documentation

Directory structure: See SDK package InstallDriver.txt

samples Compilation guide: see SDK package samples/ README.md

3 Aurora 900 SDK Call flow description

3.1 Interface call flow

Frame obtained from Stream can be obtained in two ways: active and passive.

- The image is obtained by "GetFrames" function;
- Passively obtain the image by setting the callback interface through the "RegisterFrameCb" function.

The SDK process is shown in Figure 2

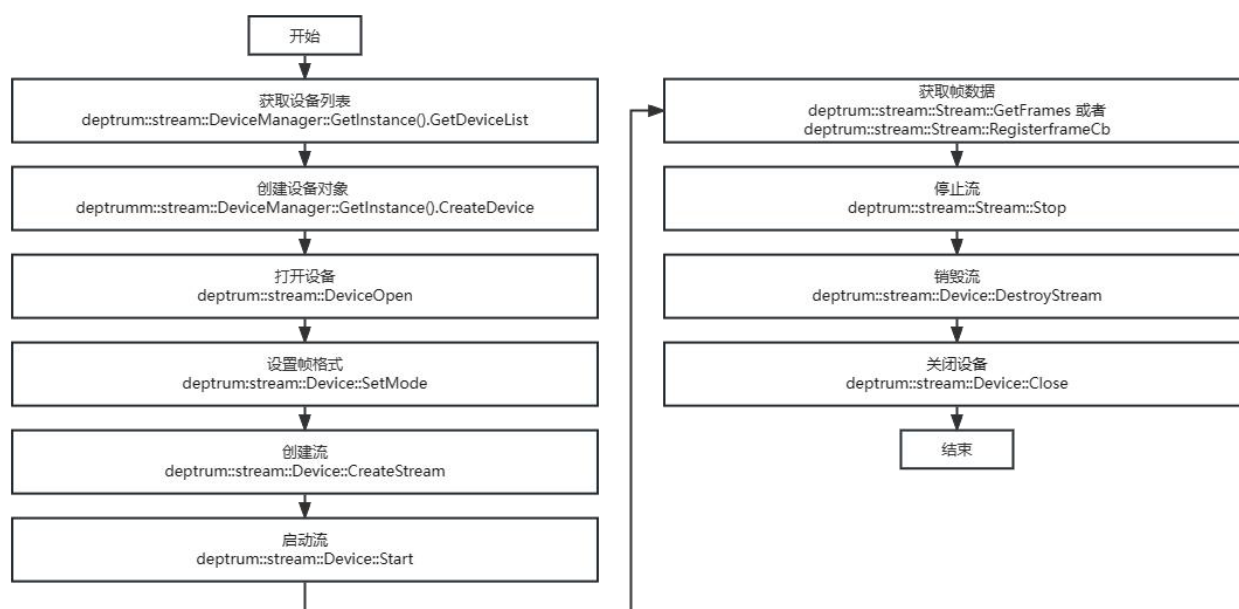


Figure 2 Interface call flow

3.2 Instructions for concurrent use

All function interfaces of this SDK are thread-safe and support multiple devices and single device multi-stream.

4 SDK usage example

4.1 Open the camera

deptrum: stream provides an interface to the Aurora 3D camera device connection system, which provides access to the device, device-related events, version and error information, and allows the creation and access of streams from the device.

```
/****** get device list *****/  
std::vector<DeviceInformation> device_list;  
int ret = GetInstance()->GetDeviceList(device_list);  
CHECK_SDK_RETURN_VALUE(ret);
```

```
/****** check device count *****/  
CHECK_DEVICE_COUNT(device_list.size());
```

```
/****** create instance & open device *****/  
std::shared_ptr<Device> device = GetInstance()-  
>CreateDevice(device_list[0]);  
CHECK_DEVICE_VALID(device);
```

```
ret = device->Open();  
CHECK_SDK_RETURN_VALUE(ret);
```

```
FrameMode ir_mode, rgb_mode, depth_mode;  
std::vector<std::tuple<FrameMode, FrameMode, FrameMode>>  
device_resolution_vec;  
device->GetDeviceSupportedFrameMode(device_resolution_vec);
```

```
ir_mode = std::get<0>(device_resolution_vec[ 0]);  
rgb_mode = std::get<1>(device_resolution_vec[ 0]);  
depth_mode = std::get<2>(device_resolution_vec[ 0]);  
ret = device->SetMode(ir_mode, rgb_mode, depth_mode);
```

```
CHECK_SDK_RETURN_VALUE(ret);
```

4.2 Create/start the stream

deptrum: Stream extracts a video stream from a device (Device), and the corresponding video frame can be obtained from the stream. For the type of stream supported by this SDK, please refer to StreamType in deptrum/ stream_ types.h under the include directory.

```
Stream* stream = nullptr;
int ret = device->CreateStream(stream, stream_type);
CHECK_SDK_RETURN_VALUE(ret);
ret = stream->Start();
```

4.3 Get the image

deptrum: Frame extracts a frame from the relevant stream. The frame type supported by this SDK is bound to the stream type, and its data format is as follows:

Table I Aurora SDK Frame data types for each Stream

Stream/Frame type	data structure
Infrared image: kIrr	uint8
Depth map: kDepth	uint16 (mm)
Infrared-deep co-extraction: kDepthIrr	uint8+ uint16 (mm)
Color image: kRgb	3 channels uint8
RGBD graph:kRgbD	RGB image + depth image
RGBD diagram-infrared same output: kRgbDIrr	RGB image + depth image + IR image
Point cloud: kPointDense	PointXYZRgbIrr<float>

There are two ways to get Frame from Stream: polling and event.

For example, call the GetFrames function to actively obtain frame data:

```
StreamFrames frames;  
ret = stream->GetFrames(frames, 2000);
```

4.4 Close the video stream

Stop the video stream and free up memory.

```
stream->Stop();  
device->DestroyStream(stream);
```

4.5 Close the camera

```
device->Close();
```

5 SDK interface specification

5.1 Header notes

header file	Content description
<code>deptrum/ device.h</code>	Equipment related public function interface
<code>deptrum/aurora900_series .h</code>	A unique functional interface for the Aurora 900 series of equipment
<code>deptrum/ stream.h</code>	Stream related function interface
<code>deptrum/stream_types.h</code>	Stream and device version related structures
<code>deptrum/stream_visualizer.h</code>	Process the depth data interface
<code>deptrum/common_types.h</code>	Internal and external related structures
<code>functional/base.h</code>	Error code, judgment return value and print calibration parameters
<code>functional/frame_rate_helper.h</code>	Print fps

5.2 Equipment module description

5.2.1 Get the device manager object

```
static std::shared_ptr<DeviceManager> GetInstance() ;
```

An instance of the device manager is obtained through this interface. The device manager object is a smart pointer type (`std::shared_ptr<DeviceManager>`) that encapsulates access to the device manager. Through this object, developers can access various functions of the device.

parameter:

not have

return:

An object of the device manager.

5.2.2 Get the device list

```
virtual int GetDeviceList(std::vector<DeviceInformation>& device_list) = 0;
```

This method is used to obtain a list of all connected devices in the current system. Developers pass a vector of DeviceInformation type (std::vector<DeviceInformation> & device_list) to this method, which fills the vector with information about each connected device in the system. The DeviceInformation structure typically includes key information such as the devices unique identifier, name, and type. The return value is an integer (int), indicating whether the operation was successful or not; a return value of 0 indicates success, while any non-zero value indicates failure.

parameter:

[out]	device_list	An array of device lists
--------	--------------------	--------------------------

return:

0: Operation successful
 Non-0: Operation failed

5.2.3 Register device plug-in callback

```
virtual void RegisterDeviceConnectedCallback(
    std::function<void(int flag, const DeviceInformation&
device_information)> handle , bool enable_hotplug
) = 0;
```

Register device plug and unplug callbacks, which allow applications to receive notifications when the device is connected or disconnected. By registering a callback function, developers can specify actions to be performed when a device plug and unplug event occurs. The flag parameter is used to identify the type of the plug and unplug event, such as connection or disconnection. The device_information parameter contains detailed information about the plugged-in device, such as the device name and type. The enable_hotplug parameter is used to enable or disable hot-plugging functionality; when set to true, the system will allow hot-plugging operations and trigger the callback function when the device is plugged in or unplugged.

parameter:

[out]	flag	1: Insert the device, 2: Pull out the device
[out]	device_information	Equipment object information
[in]	enable_hotplug	Whether to enable hot plugging

return:

5.2.4 Create a device object

```
Virtual std::shared_ptr< Device> CreateDevice(const DeviceInformation&
device_information) = 0;
```

This function is used to create a device object. When calling this function, you need to pass the details of the camera device, which are provided through the DeviceInformation structure. After successfully creating the device object, developers can further use this object to configure the camera, start preview, capture images or videos, and more.

parameter:

[in]	device_information	Equipment object information
------	---------------------------	------------------------------

return:

Returns the smart pointer to the device object

5.2.5 Create the device via the USB port

```
virtual std::shared_ptr<Device> CreateDevice ByUsbPort(std::string usb_port) = 0;
```

Create device objects directly via the USB port. This method allows developers to quickly instantiate device objects when they know the USB port connected to the device, enabling subsequent operations and controls. By specifying the USB port number, the system internally searches for and verifies devices on that port. Once it confirms that the device exists and meets the creation criteria, it returns a smart pointer to the corresponding device object.

parameter:

[in]	usb_port	The USB port number, the field described by port_path in DeviceInformation.
------	-----------------	---

return:

Returns the smart pointer to the device object.

5.2.6 Get the SDK version number

```
virtual std::string Get SdkVersion() = 0;
```

Get the version information of the SDK currently in use

return:

SDK version number

5.2.7 Get the device object type

```
virtual std::string GetDeviceName() = 0;
```

This method is used to obtain the name of the current device object, which facilitates developers to identify and manage different device objects.

parameter:

not have

return:

A string of device types.

5.2.8 Set the log output switch

```
virtual void EnableLogging (const std::string& path, bool enable)=0;
```

This feature allows developers to control whether the devices log information is output to a specified file path. When the enable parameter is true, the log information will be written to the file specified by the path parameter; when

the enable parameter is false, logging will be stopped for that file. This is very useful for debugging and monitoring the devices operational status. Developers can enable or disable logging as needed to better analyze and resolve issues.

parameter:

[in]	path	File output path
[in]	enable	File output control bit, true: output to file, false: close output

return:

not have

5.2.9 Turn on the device

```
virtual int Open()=0;
```

Open the device. Before calling other functions that depend on the devices connection status, the developer should ensure that the device has been successfully opened. The return value is an integer, usually used to indicate whether the operation is successful or not. 0 indicates success, and non-0 values indicate failure.

Parameters: None

return:

0: Operation successful
 Non-0: Operation failed

5.2.10 Get the supported image frame format

```
virtual int GetDeviceSupportedFrameMode(
    std::vector<std::tuple<FrameMode, FrameMode, FrameMode>>&
    device_resolution_vec) = 0;
```

This function is used to obtain the image frame formats supported by the device, allowing developers to select appropriate frame formats during programming. The returned device_resolution_vec parameter contains the supported frame formats. Developers can iterate through this vector to understand all the frame formats supported by the device and choose the appropriate one for subsequent development work based on actual needs. For aurora900 devices, there is currently no need to manually set the frame format; the device will automatically select an appropriate frame format based on the actual situation.

parameter:

[out]	device_resolution_vec	Storage supports frame formats
---------	------------------------------	--------------------------------

return:

0: Operation successful
 Non-0: Operation failed

5.2.11 Get the supported stream types

```
virtual int GetDeviceSupportedStreamType(std::vector<StreamType>&
device_streamtype_vec)
= 0;
```

Obtain information about the stream types supported by the device. The device_streamtype_vec parameter is a vector used to store all stream types supported by the device. Developers can iterate over this vector to understand all the stream types supported by the device, allowing them to select appropriate stream types based on actual needs in subsequent development work.

parameter:

[out]	device_streamtype_vec	Supported stream types
--------	------------------------------	------------------------

return:

0: Operation successful
Non-0: Operation failed

5.2.12 Set the IR frame rate

```
virtual int SetIrFps(int ir_fps) = 0;
```

Set the IR frame rate of the device. Note that the RGB frame rate will also take effect synchronously. The ir_fps parameter allows developers to specify the specific value of the IR frame rate.

parameter:

[in]	ir_fps	The frame rate of the IR is aurora930, which only supports 5, 10,12, and 15 fps
-------	---------------	---

return:

0: Operation successful
Non-0: Operation failed

5.2.13 Set the device image frame format

```
virtual int SetMode(const FrameMode& ir_mode= kInvalid,
                    const FrameMode& rgb_mode= kInvalid,
                    const FrameMode& depth_mode = kInvalid,
                    FrameDecodeMode ir_decode_mode = kSoftwareDecode,
                    FrameDecodeMode rgb_decode_mode = kSoftwareDecode) = 0;
```

Since frame format is not required for aurora900 devices at present, you can directly call the SetMode() method and use the default value.

parameter:

[in]	ir_mode	I R image frame format
[in]	rgb_mode	R GB image frame format
[in]	depth_mode	Depth image frame format
[in]	ir_decode_mode	IR image frame decoding mode



```
return:
    0: Operation successful
    Non-0: Operation failed
```

```
virtual int Close() = 0;
```

Parameters: None

```
return:
    0: Operation successful
    Non-0: Operation failed
```

```
virtual int Reboot() = 0;
```

```
return:
    0: Operation successful
    Non-0: Operation failed
```

```
virtual int Upgrade(const std::string& update_file_path,  
                   ProgressHandler progress_handler = nullptr) = 0;
```

parameter:

[in]	update_file_path	Update the path of the file
[in]	progress_handler	Return the upgrade progress through the callback interface.

return:

0: Operation successful
Non-zero: Operation failed

5.2.17 Create a stream object

```
virtual int CreateStream(Stream*& stream, const std::vector<StreamType>& types) = 0;
```

The function to create stream objects allows users to specify different types of streams according to their needs. The Stream* parameter is a pointer to the stream object, through which you can access and manipulate the created stream. const std: vector<StreamType> & types parameter is a vector containing the required stream types, allowing users to add one or more stream types to this vector as needed. The device driver will attempt to create one or more stream objects based on the provided types and assign their pointers to the stream parameter. If creation is successful, the function returns 0 indicating a successful operation; if creation fails, it returns a non-zero value indicating an error.

parameter:

[in/out]	stream	A reference to an object pointer
[in]	types	The type of stream can be multiple

return:

0: Operation successful
Non-0: Operation failed

pay attention to:

A reference to an object pointer

5.2.18 Destroy the object

```
virtual int DestroyStream(Stream*& stream) = 0;
```

Destroy a stream object that has already been created to release resources, and destroy a stream object of a specified type. When destroying a stream object, the resources associated with that stream object are released.

parameter:

[in]	stream	A reference to an object pointer
------	---------------	----------------------------------

return:

0: Operation successful
Non-0: Operation failed

5.2.19 Rgbdir Align switch (only supports AURORA930)

```
virtual int SwitchAlignedMode(bool mode = false) = 0;
```

The Rgbdir align switch is used to adjust the RGBDIR data alignment method of the AURORA930 device. When the

SwitchAlignedMode function is called with the true parameter, the device will enable the align mode; passing false as the parameter will turn off the align mode. Calling this function after SetMode ensures that the devices current state has been configured according to expectations before switching to the align mode.

parameter:

[in]	mode	Set false by default, and true is to enable alignment
-------	-------------	---

return:

0: Operation successful
 Non-0: Operation failed

5.2.20 Set the threshold size for noise removal

```
virtual int SetRemoveFilterSize (uint32_t size = 110) = 0;
```

Used to set which noise points will be removed during the image processing. Setting an appropriate threshold size can effectively reduce noise interference in images while retaining important details. If the threshold is set too low, it may misidentify and remove some important image details as noise; conversely, if the threshold is set too high, it may fail to effectively remove noise from the image. The threshold for removing noise can be reasonably set according to the actual application scenario and requirements, and can be called after SetMode.

parameter:

[in]	Size	The default is 110
-------	-------------	--------------------

return:

0: Operation successful
 Non-0: Operation failed

5.2.21 Obtain the serial number of the device product

```
virtual int GetSerialNumber(std::string *sn) = 0;
```

This interface is used to obtain the unique product serial number of the current device for identification and management of the device. After calling this interface, the product serial number of the device will be stored in the incoming string pointer parameter.

parameter:

[out]	sn	Product serial number
--------	-----------	-----------------------

return:

0: Operation successful
 Non-0: Operation failed

5.2.22 Obtain the internal and external parameters of the camera

```
virtual int GetCameraParameters(Intrinsic& ir_intri, Intrinsic& rgb_intri,
Extrinsic& ext) = 0;
```

This interface is used to obtain the intrinsic and extrinsic parameters of the current devices camera. The intrinsic parameters include the focal length, principal point coordinates, and other internal parameters of the camera, while the extrinsic parameters describe the position and orientation of the camera relative to a reference coordinate system. By obtaining these parameters, images captured by the camera can be accurately corrected and positioned. After calling this interface, the intrinsic parameters of the camera will be stored in the incoming `ir_intri` and `rgb_intri` parameters, and the extrinsic parameters will be stored in the incoming `ext` parameter.

parameter:

[out]	ir_intri	I r Camera internal parameters
[out]	rgb_intri	Internal parameters of RGB camera
[out]	ext	Camera external parameters

return:

0: Operation successful
Non-0: Operation failed

5.2.23 Get device information

```
virtual void GetDeviceInfo(DeviceInformation& device_information)  
= 0;
```

This interface is used to obtain detailed information about the device, including the address, manufacturer, name, port number, etc. After calling this interface, the device information will be filled into the incoming `device_information` parameter.

parameter:

[out]	device_information	facility information
--------	---------------------------	----------------------

return:

not have

5.2.24 Restart the device

```
virtual int RestartDevice(const DeviceInformation device_information)  
= 0;
```

This interface is mainly used to restart the device and restore the stream by passing in device information parameters when the device is abnormal or needs to be restarted. In general, it is not recommended to call it actively.

parameter:

[in]	device_information	facility information
-------	---------------------------	----------------------

return:

0: Operation successful
Non-0: Operation failed

5.2.25 Stop the device

```
virtual int StopDevice( const DeviceInformation& device_information
) = 0;
```

After calling this interface, the device will stop stream and shut down the device. It is not recommended to call it actively.

parameter:

[in]	device_information	facility information
-------	---------------------------	----------------------

return:

0: Operation successful
Non-0: Operation failed

5.2.26 Get the device status

```
virtual bool GetDeviceStatus() = 0;
```

This interface is used to obtain the status information of the current device. The return value is a Boolean type. If true is returned, it indicates that the device status is successfully obtained; if false is returned, it may indicate that the device status is failed or the device cannot provide status information at present.

Parameters: None

return:

True: Enable the device
False: The device is off

5.2.27 Get device version information

```
virtual int GetDeviceInfo (DeviceDescription& device_info
) = 0;
```

Obtain device information, including:

device_name represents the device name.

serial_num is the serial number of the device, which is used to uniquely identify the device.

stream_sdk_version indicates the version number of the SDK.

rgb_firmware_version and ir_firmware_version respectively represent the version number of RGB and infrared firmware.

Pid (Product ID) and vid (Vendor ID) are the hardware identifiers of the device.

parameter:

[out]	device_info	For product version information, refer to DeviceDescription
-------	--------------------	---

return:

0: Operation successful
Non-0: Operation failed

5.2.28 Deep/infrared image mirroring (aurora900 equipment available)

```
virtual int EnableDepthIrMirroring(bool enable) = 0;
```

Mirror depth images and infrared images can be enabled or disabled by developers through the call to the EnableDepthIrMirroring method. When the enable parameter is true, depth images and infrared images will be mirrored, meaning the image content is flipped horizontally; when the enable parameter is false, the mirror function is turned off, and the images remain in their original state.

parameter:

[in]	enable	Whether to enable deep mirror true to enable and false to disable
--------	---------------	---

return:

0: Operation successful
Non-0: Operation failed

5.2.29 Softreset device

```
virtual int SoftResetDevice() = 0;
```

In some cases, the device may need to perform a software reset operation to restore its initial state or resolve operational anomalies. The SoftResetDevice method provides a convenient way to achieve this goal. A return of 0 indicates that the software reset operation was successfully completed; a non-zero value indicates that the operation failed.

Parameters: None

return:

0: Operation successful
Non-0: Operation failed

5.2.30 Set the automatic exposure mode

```
virtual int SetAEMode(const CameraComponent& component, uint8_t mode) = 0;
```

Set the automatic exposure mode, which allows users to specify the camera component and the desired exposure mode. Parameter component specifies the camera component for which the exposure mode is to be set, while the mode parameter defines the specific exposure mode. A return value of 0 indicates successful setting of the automatic exposure mode; any non-zero value indicates a failure in the operation.

parameter:

[in]	component	Types of sensors
[in]	mode	Automatic exposure mode

return:

0: Operation successful
Non-0: Operation failed

5.2.31 Get the automatic exposure mode

```
virtual int GetAEMode(const CameraComponent& component, uint8_t& mode) = 0;
```

This method is used to retrieve the current auto exposure mode of a specified camera component. Parameter component specifies the camera component for which the exposure mode is queried, and the mode parameter is passed by reference to store the retrieved exposure mode value. A return value of 0 indicates successful retrieval of the auto exposure mode; any non-zero value indicates an operation failure.

Note: speckle is excluded.

parameter:

[in]	component	Types of sensors
[out]	mode	Automatic exposure mode

return:

0: Operation successful
 Non-0: Operation failed

5.2.32 Switch of the automatic exposure function

```
virtual int SwitchAutoExposure(bool enable) = 0;
```

This method is used to control the automatic exposure function of the camera. Parameter enable is a Boolean type, which is used to specify whether to enable the automatic exposure function. When enable is true, the automatic exposure function is enabled; when enable is false, the automatic exposure function is disabled.

parameter:

[in]	enable	True enables IR automatic exposure, False turns off IR automatic exposure, and it needs to be turned off before manually setting the exposure parameters.
--------	---------------	---

return:

0: Operation successful
 Non-0: Operation failed

5.2.33 Set the exposure time

```
virtual int SetExposure(const CameraComponent& component, int exposure_in_us)=0;
```

This interface allows users to set the exposure time for a specified camera component. Parameter component represents the identifier of the camera component, and exposure_in_us represents the exposure time in microseconds. The range for setting the exposure time is between 1 and 31, with each value representing an exposure time of 0.1 milliseconds. For example, if you need to set the exposure time to 2.5 milliseconds, you should pass 25 as the value for exposure_in_us.

Please note, before calling this interface, users should ensure that the auto exposure function has been correctly disabled; otherwise, the set exposure time may be overwritten by the auto exposure algorithm. Additionally, the setting of exposure time may be limited by the camera module hardware. If the set exposure time exceeds the hardware support range, it may result in operation failure or abnormal camera behavior.

A return value of 0 indicates that the operation is successful, and a non-0 value indicates that the operation fails. After calling this interface, users can obtain the current exposure time setting through the corresponding interface to confirm whether the setting is successful.

explain:

parameter:

[in]	component	Types of sensors
[in]	exposure_in_us	exposure time

return:

0: Operation successful
Non-0: Operation failed

5.2.34 Get the exposure time

```
virtual int GetExposure(const CameraComponent& component, int&
exposure_in_us)=0;
```

The exposure time of the specified sensor in the acquisition module is obtained. speckle graph is not supported for the time being.

parameter:

[in]	component	Types of sensors
[out]	exposure_in_us	exposure time

return:

0: Operation successful
Non-0: Operation failed

5.2.35 Set sensor gain

```
virtual int SetGain(const CameraComponent& component, uint32_t gain) = 0
```

This function is used to adjust the gain value of a specified sensor in the camera module to improve image quality. The higher the gain value, the brighter the image, but the noise may also increase accordingly.

parameter:

[in]	component	Types of sensors
[in]	gain	The value of the gain

return:

0: Operation successful
Non-0: Operation failed

5.2.36 Get sensor gain

```
virtual int GetGain(const CameraComponent& component, uint32_t& gain) = 0
```

This function is used to read the current gain value of a specified sensor in the camera module. By calling this function, users can obtain the gain value set by the current sensor for further analysis or image quality adjustment.

parameter:

[in]	component	Types of sensors
[out]	gain	The value of the gain

return:

0: Operation successful
 Non-0: Operation failed

5.2.37 Set indoor and outdoor mode (only support AURORA930)

```
virtual int SetLaserDriver(float laser_driver) = 0
```

This function allows users to set the camera module to indoor or outdoor modes. In different lighting conditions, these modes can help optimize the cameras image capture performance. For example, in strong outdoor light, the camera may adjust its exposure and gain settings to reduce overexposure and preserve image details; in low indoor light, the camera may increase gain to enhance image brightness. By calling this function, users can select the appropriate mode based on the current environment, thereby improving image quality and camera adaptability.

parameter:

[in]	laser_driver	Indoor and outdoor mode, 1-auto 2-indoor 3-outdoor
-------	---------------------	--

return:

0: Operation successful
 Non-0: Operation failed

5.2.38 Set the depth filter range (only support AURORA930)

```
virtual int FilterOutOfRangeDepthMap(int min = 150, int max = 4000) = 0
```

This function allows users to filter out the depth data outside the set depth range and keep only the data within the specified depth range. By adjusting the minimum and maximum depth values, users can precisely control the depth data range, so as to optimize subsequent 3D reconstruction, object recognition and other applications.

parameter:

[in]	min	Minimum depth range
[in]	max	Maximum depth range

return:

0: Operation successful
 Non-0: Operation failed

5.2.39 Switch of depth correction (only supported by AURORA930)

```
virtual int DepthCorrection(bool depth_correction) = 0
```

This function is used for the depth data calibration function of the switch device. By enabling the depth correction function, the device can automatically adjust the error in the depth data, so that the generated 3D image and object recognition results are more accurate.

parameter:

[in]	depth_correction	True: Enable depth calibration False: Disable depth calibration
--------	-------------------------	---

return:

0: Operation successful
Non-0: Operation failed

5.2.40 Set the face exposure area

```
virtual int SetFaceArea(DeptrumBBBox face_area) = 0;
```

This function allows users to customize the exposure area of the face. By adjusting the coordinate parameters, the brightness and contrast of the face in the image can be optimized, so as to improve the accuracy and stability of face recognition.

parameter:

[in]	face_area	Face exposure area
--------	------------------	--------------------

return:

0: Operation successful
Non-0: Operation failed

5.2.41 Get information about device support

```
virtual int GetSupportInfo(SupportedInfo& support_info) = 0;
```

The device can support relevant functional information, such as:

scan_face_mode: Indicates whether the device supports face scanning mode. 1 indicates support, and 0 indicates no support.

scan_code_mode: Indicates whether the device supports barcode scanning mode. 1 indicates support and 0 indicates no support.

running_7x24_hours: Indicates whether the device supports 7x24 hours of uninterrupted operation. 1 means support and 0 means no support.

Depth range: indicates the depth range information of the device, including specific parameters such as minimum depth and maximum depth.

is_support_se: Indicates whether the device supports secure encryption. 1 indicates support, and 0 indicates no support.

is_support_synced_2_img: indicates whether the device supports the function of synchronizing two images. 1 means support and 0 means no support.

parameter:

[out]	support_info	Supported information content
---------	---------------------	-------------------------------

return:

0: Operation successful
Non-0: Operation failed

5.2.42 Get temperature information

```
virtual int GetCameraTemperature(TemperatureType temp_type, int16_t* temperature) = 0;
```

The temperature information of each module inside the module can be obtained. Currently, the motherboard temperature, laser temperature and CPU temperature are supported

parameter:

Access temperature module:		
[in]	temp_type	TemperatureCamera = 1, TemperatureVcsel , TemperatureCpu ,
[out]	temperature	Module temperature

return:

0: Operation successful
Non-0: Operation failed

5.2.43 Rollback firmware version

```
virtual int RollBack() = 0;
```

This feature allows users to roll back the firmware version of their device to the previous one. In some cases, if the current firmware version has instability or compatibility issues, rolling back the firmware version may be a viable solution. By calling the RollBack function, users can trigger the rollback process. Please note that it is recommended to back up the current firmware before performing the rollback operation to prevent data loss or device malfunction. After the rollback operation is complete, the device will restart and load the previous firmware version.

parameter:

return:

0: Operation successful
Non-zero: Operation failed

5.2.44 Set the depth of the edge trimming size

```
virtual int SetFilterOutBorderDepth(int border_depth) = 0;
```

This feature allows users to adjust the size of the edge trimming when performing depth measurements. Edge trimming refers to the process where the device ignores data from the edge areas during depth measurements to reduce measurement errors caused by edge effects. By setting the `border_depth` parameter, users can control the size of the ignored edge areas.

parameter:

[in]	border_depth	The depth of the cut is [0-15]
-------	---------------------	--------------------------------

return:

0: Operation successful
Non-0: Operation failed

5.2.45 Set the depth distance

```
virtual int SetDepthRange(int min_range, int max_range) = 0;
```

This feature allows users to define the minimum and maximum measurement range of the device when performing depth measurements. By setting the `min_range` and `max_range` parameters, users can limit the device to only measure data within a specific depth range.

parameter:

[in]	min_depth	Minimum depth range
[in]	max_depth	Maximum depth range

return:

0: Operation successful
Non-0: Operation failed

5.2.46 Set the heartbeat

```
virtual int SetHeartbeat(const HeartbeatParam& heartbeat_params,  
HeartbeatResult heartbeat_callback = nullptr) = 0;
```

This function is used to configure the devices heartbeat mechanism, allowing it to send heartbeat signals periodically to maintain a connection with the device or monitor its status. The `HeartbeatParam` structure contains parameters for configuring heartbeat signals, such as heartbeat interval and data content of the heartbeat signal. `heartbeat_callback` is a callback function that is called when the device receives a heartbeat signal and can handle the heartbeat signal or perform related operations. If there is no need to process the heartbeat signal, the `heartbeat_callback` parameter can be set to `nullptr`. By setting up the heartbeat mechanism, users can ensure the stability of the devices connection and promptly detect any abnormal conditions.

explain:

For devices that do not support heartbeat, if enabled, a heartbeat failure callback will be received.

parameter:

[in]	heartbeat_params	Heartbeat parameters, including timeout time, number of allowed failures, and whether to enable
[in]	heartbeat_callback	Heartbeat caught up

return:

0: Operation successful
Non-0: Operation failed

5.2.47 Stop the heartbeat

```
virtual int StopHeartbeat() = 0;
```

When the device no longer needs the heartbeat mechanism for connection stability detection, the StopHeartbeat function can be called to stop the heartbeat. This function does not accept any parameters; after execution, it will shut down the current heartbeat mechanism and stop sending and receiving heartbeat signals. After stopping the heartbeat, the device will no longer trigger the heartbeat callback, and users will no longer be able to detect the device's connection status through the heartbeat mechanism.

return:

0: Operation successful
Non-0: Operation failed

5.2.48 Set it to scan code mode

```
virtual int EnableScanCodeMode (bool enable) = 0;
```

When you need to set the device to scan mode, you can call the EnableScanCodeMode function. The function accepts a Boolean parameter enable to specify whether to enable scan mode. When enable is true, the device will enter scan mode.

parameter:

[in]	enable	True: enable scan code mode, False: disable scan code mode
--------	---------------	--

return:

0: Operation successful
Non-0: Operation failed

5.2.49 Set the image resolution

```
virtual int SetStreamImageResolution (StreamImageResolution  
image_resolution) = 0;
```

Set the image resolution

The image resolution can be set by calling this function. StreamImageResolution is an enumeration type that defines the different image resolution options supported by the device.

pay attention to:

Currently, only rgb image resolution adjustment is supported. The following resolutions are supported:
768x432/1920x1080

parameter:

[in]	image_resolution	image resolution ratio
--------	-------------------------	------------------------

return:

0: Operation successful
Non-zero: Operation failed

5.2.50 Get the laser current

```
virtual int GetLaserCurrent (int& current) = 0;
```

This function is used to obtain the operating current of the laser, in mA.

parameter:

[out]	current	current value (1800mA, 1500mA, 1600mA, 1700mA, 900mA, 2000mA, 1000mA, 1100mA, 1200mA, 1300mA, 1400mA, 500mA, 600mA, 700mA, 800mA, 900mA, 1450mA), the default value is 1450mA
---------	----------------	--

return:

0: Operation successful
Non-0: Operation failed

5.2.51 Set the laser current

```
virtual int SetLaserCurrent ( int current) = 0;
```

This function is used to set the operating current of the laser, measured in mA. Users can call this function to adjust the lasers operating current to their desired value. When calling this function, users need to pass an int parameter current, which specifies the new operating current value for the laser. If the function executes successfully, the lasers operating current will be set to this value. If the function fails, it will return a non-zero value indicating an error. By adjusting the lasers operating current, users can control the output power of the laser, thereby meeting different application requirements.

parameter:

[in]	current	current value (1800mA, 1500mA, 1600mA, 1700mA, 900mA, 2000mA, 1000mA, 1100mA, 1200mA, 1300mA, 1400mA, 500mA, 600mA, 700mA, 800mA, 900mA, 1450mA), the default value is 1450mA
--------	----------------	--

return:

0: Operation successful
Non-0: Operation failed

pay attention to:

After setting the current, you need to restart it to take effect

5.2.52 Set Laser/Led light mode

```
virtual int SetLightMode(StreamLightMode light_mode) = 0;
```

This function is used to set the Laser/Led Light Switch,
The switch mode of the Laser and Led can be controlled by calling this function.

parameter:

[in]	light_mode	Light mode;
		1: all on; 2: all off; 3: laser on, led off; 4: laser off, led on

return:

0: Operation successful
Non-0: Operation failed

5.3Stream module description

5.3.1 Start the stream

```
virtual int Start() = 0;
```

Used to start the camera data stream. After calling this function, the system will begin to collect and transmit camera data. If the function call is successful, it will return 0, indicating that the camera data stream has been successfully started. Users can proceed with subsequent data processing or analysis based on this. Please ensure that the camera device is correctly connected and initialized before calling this function.

Parameters: None

return:

0: Operation successful
Non-0: Operation failed

5.3.2 Stop streaming

```
virtual intStop() = 0;
```

After calling this function, the system will stop collecting and transmitting the data of the camera. If the function call is successful, it will return 0, indicating that the camera data stream has been successfully stopped. After stopping the data stream, users can perform other operations, such as shutting down the camera device or releasing related resources.

Parameters: None

return:

0: Operation successful
Non-0: Operation failed

5.3.3 Get single/multi-frame images

```
virtual int GetFrames( StreamFrames& frames, uint32_t timeout = -1) = 0;
```

This function is used to obtain single or multiple frames of image data from a camera device. The user can specify an timeout (timeout), measured in milliseconds. If the image data is not obtained within the specified timeout, the function will return failure. If no timeout is set (i.e., timeout is the default value-1), the function will block indefinitely until the image data is successfully acquired.

StreamFrames The structure is used to store the acquired image frame data, including the number of image frames, the width and height of each frame image, pixel format and other information, as well as the actual storage location of the image data. Users can access this structure to obtain and process the image data.

Note that during the call to the GetFrames function, the RegisterFrameCb function should not be called simultaneously to register the image data callback. The two are mutually exclusive, and simultaneous calls may result in undefined behavior or errors.

parameter:

[out]	frame s	Frame s Reference to an object
[in]	timeout	The timeout time is in milliseconds

return:

0: Operation successful
Non-0: Operation failed

5.3.4 Register the callback interface

```
virtual void RegisterFrameCb(std::function<int( StreamFrames&)> cb) = 0;
```

Active registration of callback interface (exclusive to GetFrames interface)

The active registration callback interface allows users to automatically obtain and process the image frame data when it arrives, without having to call the GetFrames function.

When the registration callback interface is set up, whenever new image frame data arrives, the system automatically calls the user-provided callback function and passes the StreamFrames structure containing the image frame data as a parameter to this function. The callback function can process the image data as needed, such as displaying, saving, or further analyzing it.

Note that because the callback interface and GetFrames interface are mutually exclusive, you should not call the GetFrames function to retrieve image data after registering the callback interface. Otherwise, undefined behavior or errors may occur.

parameter:

[in]	cb	A callback function pointer
-------	-----------	-----------------------------

return:

0: Operation successful
Non-0: Operation failed

6 Structure and enumeration type description

6.1 Basic type

6.1.1 Stream type

```
enum StreamType {  
    kInvalidStreamType = 0,    // Invalid stream  
    kRgb = 1,                  // Rgb stream  
    kIr = 2,                   // Ir stream  
    kDepth = 3,                // Depth stream  
    kRgbd = 4,                 // Rgbd stream, frames order in rgb depth  
    kRgbIr = 5,                // RgbIr Stream, frames order in rgb ir  
    kDepthIr = 6,              // Depthir stream, frames order in depth ir  
    kRgbdIr = 7,               // Rgbd Ir stream, frames order in rgb depth ir  
    kDepthIrLaser = 8,  
    kSpeckleCloud = 9,  
    kPoint Cloud = 10,         // Dense point cloud stream  
    kRgbdPointDense = 11,      // Rgbd Dense point cloud stream  
    kRgbdIrPointDense = 12,    // Rgbd Ir Dense point cloud stream;  
    kRgbdIrFlag = 13,          //rgbdir+flag stream  
}
```

6.1.2 frame type

```
enum FrameType {  
    kInvalidFrameType = 0,      // Invalid frame  
    kRgbFrame = 1,              // RGB frame  
    kIrFrame = 2,               // IR frame  
    kDepthFrame = 3,            // Depth frame  
    kLaserFrame = 4,            // flag frame  
    kPoint CloudFrame = 5,      // Dense point cloud frame  
    kRgbdPointCloudFrame = 6,   // Rgbd point cloud frame  
    kRgbdIrPointCloudFrame = 7, // Rgbd Ir point cloud frame  
}
```

6.1.3 A single frame

```
struct StreamFrame {  
    int index;                  // Index of the frame  
    int size;                   // Frame data size in bytes  
}
```

```
int cols;                // Number of columns
int rows;                // Number of rows
int bits_per_pixel;      // Number of bits per pixel
float temperature;       // Driver temperature during this frame
FrameType frame_type;    // Type of the frame, refer to FrameType
FrameAttr frame_attr;    // Attributes of the frame, refer to FrameAttr
ImageFormat image_format; // Format of the image
uint64_t timestamp;      // Timestamp of the frame
std::shared_ptr<void> data; // Pointer to the frame data
};
```

parameter:

frame_type	frame type
index	Frame index
size	Data size
cols	width
rows	altitude
bits_per_pixel	Each bit of precision
temperature	driver temperature
frame_attr	Frame property information
data	data pointer
timestamp	time stamp

remarks

The data type for each pixel in the depth and ir streams is uint 16_t, where depth is measured in mm; the format of the depth-ir stream data is: depth image + ir image, with the same format as above; the coordinate type for each point in the point-cloud stream is float, with units in mm; the color component data type for each pixel in the rgb stream is uint_8, in the order B GR. For detailed parsing methods, see samples.

6.1.4 Multi frames

```
struct StreamFrames {
int count;                // Number of frames
std::vector<std::shared_ptr<StreamFrame>> frame_ptr; // Pointer to the frames
array
std::shared_ptr<ExtraFrameInfo> extra_info{};        // Extra frames
information
};
```

parameter:

count	Number of frames
frame_ptr	Frame pointer
extra_info	extend information

6.1.5 facility information

```
struct DeviceDescription{
    std::string device_name;
    std::string serial_num;
    std::string stream_sdk_version;
    std::string rgb_firmware_version;
    std::string ir_firmware_version;
    uint16_t pid;
    uint16_t vid;
};
```

parameter:

device_name	Product model name
serial_num	Product serial number
stream_sdk_version	Stellar SDK Version number
rgb_firmware_version	Rgb camera firmware version
ir_firmware_version	Tof camera firmware version
pid	Tof camera product ID
vid	Tof camera manufacturer ID

6.2 Device type

6.2.1 Camera component type

```
enum CameraComponent {
    kInvalidCameraComponent = 0,
    kIrCamera = 0,
    kRgbCamera = 1,
    kLaser = 2,
    kLed = 3,
};
```

parameter:

kIrCamera	IR camera
kRgbCamera	RGB camera
kLaser	laser
kLed	infrared

6.2.2 Camera component ID

```
struct CameraComponentID {
    uint8_t device_addr = 0;
    uint16_t vid = 0;
    uint16_t pid = 0;
    std::string serial_number;
```

```
std::string name;
std::string port_path;
};
```

parameter:

device_addr	device address
vid	vender ID
pid	product ID
serial_number	Device serial number
name	Camera name
port_path	The U SB port address

6.2.3 Facility information

```
struct Device Information {
    std::string model;
    CameraComponentID ir_camera;
    CameraComponentID rgb_camera;
};
```

parameter:

model	product model
ir_camera	infra-eye ID
rgb_camera	RGB camera I D

6.2.4 Version information

```
struct Version {
    uint8_t major;
    uint8_t minor;
    uint8_t revision;
    char tag[8];
};
```

parameter:

major	The main version number
minor	Sub-version number
revision	Update the version number
tag[8]	tab

6.2.5 Frame format

```
struct FrameFormat {
    int rows;
    int cols;
    int frame_size;
    int bits_per_pixel;
};
```

parameter:

rows	line number
cols	column number
frame_size	Data size
bits_per_pixel	The size of each pixel

6.2.6 Camera resolution (RGB camera)

```
enum StreamImageResolution{
    kResolutionInvalid = 0,
    kResolution480x768,
    kResolution720x1280,
    kResolution1080x1920,
};
```

parameter:

kResolutionInvalid	Invalid resolution
kResolution480x768	480x768
kResolution720x1280	720x1280
kResolution1080x1920	1080x1920

6.3 Depth calculation type

6.3.1 Camera intrinsic parameters

```
struct Intrinsic {
    int rows;
    int cols;
    float focal_length[2];
    float principal_point[2];
    float distortion_coeffs[5];
};
```

parameter:

rows	line number
cols	column number
focal_length[2]	Focal length of lens
principal_point[2]	principal point

<code>distortion_coeffs[5]</code>	Camera distortion parameters
-----------------------------------	------------------------------

6.3.2 Camera Extrinsic parameters

```
struct Extrinsic {
    float rotation_matrix[9];
    float translation_vector[3];
};
```

parameter:

<code>rotation_matrix[9]</code>	spin matrix
<code>translation_vector[3]</code>	Translation matrix

6.3.3 Spatial point coordinates

```
template<class T>
struct PointXYZ {
    T x, y, z;
};
```

parameter:

<code>T</code>	Coordinate data type
<code>x</code>	x coordinate
<code>y</code>	y coordinate
<code>z</code>	z coordinate

6.3.4 Point cloud coordinates and corresponding rgb and ir data

```
template<class T>
struct PointXYZ R gb Ir {
    T x, y, z;
    uint8_t r, g, b, ir;
};
```

parameter:

<code>T</code>	Coordinate data type
<code>x</code>	x coordinate
<code>y</code>	y coordinate
<code>z</code>	z coordinate
<code>r</code>	Red color component
<code>g</code>	Green color component
<code>b</code>	b Labeled color components
<code>i r</code>	ir data

Amendments to records

edition	description	date	expurgator
V 1	Initial	15/03/20 22	Zhang
V2	Initial	5/9/2023	Yin
V3	Add missing interface description	5/29/2023	Yins
V4	Heart function, GetFrames update	19/04/2024	Qin
V5	Add EnableScanCodeMode and SetStreamImageResolution interfaces	27/08/2024	Wu
V6	Add a setting/get laser current interface	12/18/2024	Wu
V7	Optimize the function description and modify the document format	03/04/2025	Song

Disclaimer

The device application information and other similar content described in this publication are provided for your convenience only and may be superseded by updated information. Ensuring that applications comply with technical specifications is your own responsibility. Shenzhen Guangjian Technology Co., Ltd. makes no express or implied, written or oral, statutory or other representations or warranties regarding these information, including but not limited to any representation or warranty concerning its use, quality, performance, suitability for particular purposes, or applicability. Guangjian Technology assumes no responsibility for any consequences arising from this information or its use. Without the written approval of Shenzhen Guangjian Technology Co., Ltd., the products of Guangjian Technology shall not be used as critical components in life support systems. Under the protection of Guangjian Technologys intellectual property rights, no license shall be transferred secretly or otherwise.

Technical support

You can get support through the following channels:

- Website support: Visit www.deptrum.com for relevant documentation and online support
- Email support: please send your specific questions to support@deptrum.com
- FAE support: Please contact our sales staff for FAE support